Claude API Integration

Smart Onboarding with URL Analysis and Conditional Follow-up Questions

Prepared for: Muhammad Ismaeel | Stormo.io | June 2026 | Hardball Ventures LLC

This document specifies how to integrate the Claude API into Stormo's onboarding flow in two ways. First, when a merchant provides their store URL in Topic 1, the system fetches and analyzes it with Claude to understand the store's positioning and products. Second, Claude uses context from all merchant answers to parse free text responses and extract key data points. Only follow-up questions for missing information are displayed, making the onboarding feel conversational and intelligent. This is a launch blocker feature.

LAUNCH BLOCKER: This feature must be implemented before the first merchant-facing release. URL analysis immediately provides Claude with store context. Conditional follow-ups make the onboarding intelligent. Together they define the first impression of Stormo's AI quality.

1. Architecture Overview

1.1 Two-Part Claude Integration

The onboarding uses Claude API in two distinct ways:

Part 1: Store URL Analysis (Topic 1)

When the merchant enters their store URL, the system immediately fetches the homepage and passes it to Claude for analysis. Claude examines the store and returns context: what they sell, their positioning, product categories, price tier, unique selling points. This context informs every follow-up question in the rest of the onboarding.

Part 2: Answer Parsing (All Topics)

After the merchant answers any free text question, Claude parses that answer to extract key data points. Only follow-up questions for missing information are displayed. A merchant who provides complete answers skips follow-ups. A merchant who provides thin answers gets gentle guidance.

1.2 Why This Matters

- URL analysis makes Claude immediately knowledgeable about the actual store, not just the merchant's description.
- Follow-up questions can be tailored based on what Claude found at the URL, not generic.
- Merchants who describe their store completely skip redundant follow-ups.
- First impression of Stormo's AI intelligence is formed in Topic 1 and reinforced throughout.

1.3 Cost and Performance

URL Fetch Cost: Minimal. A single HTTP GET to the homepage is one request, <5KB of data typically.

Claude API Calls: Approximately 6-7 calls per onboarding. URL analysis plus answer parsing for 5 topics. At $3 per million input tokens, cost per merchant is negligible.

Latency: URL fetch adds 200-500ms. Claude calls add 200-500ms each. Total added latency to onboarding approximately 2-3 seconds. Acceptable.

2. URL Analysis Feature

2.1 How URL Analysis Works

Step 1: Merchant enters their store URL in Topic 1

Step 2: Frontend validates the URL format and reachability

Step 3: Backend fetches the homepage HTML

Step 4: Backend sends the HTML to Claude with analysis prompt

Step 5: Claude returns structured analysis: product categories, positioning, price tier, niche, unique elements

Step 6: Frontend displays the analysis and uses it to customize follow-up questions

2.2 What Claude Extracts from URL Analysis

- Primary product categories and what they sell
- Positioning: luxury, budget, mid-market, boutique, mass market
- Price tier: sub-$25, $25-75, $75-150, $150+
- Store stage: new and empty, early products, established catalog
- Unique value propositions: handmade, organic, vintage, sustainable, etc.
- Visual aesthetic: minimalist, maximalist, trendy, classic, artisanal
- Target audience signals: age demographic, gender, lifestyle

2.3 Using URL Analysis in Follow-ups

After Claude analyzes the store, the onboarding becomes smarter. Examples:

If Claude detects handmade leather goods: Ask about production capacity, turnaround times, material sourcing instead of generic product questions.

If Claude detects luxury positioning: Ask about target income level, brand storytelling approach instead of treating it like a general store.

If Claude detects a new store with few products: Ask about expansion plans and product roadmap.

2.4 Backend Implementation

Create a new endpoint POST /api/onboarding/analyze-store-url that:

- Accepts the store URL
- Validates the URL format
- Fetches the homepage using Node.js fetch or axios
- Extracts text content from the HTML using cheerio or similar
- Sends the extracted content to Claude with a structured analysis prompt
- Returns the analysis as JSON
- Gracefully handles unreachable URLs, invalid formats, timeouts

2.5 Error Handling for URL Analysis

If the URL is unreachable or invalid:

- Display a user-friendly error message: The store URL could not be reached. Please check the URL and try again.
- Allow the merchant to try again or skip URL analysis
- If skipped, proceed with onboarding but show all follow-up questions as a fallback
- Log the error for debugging but never crash the onboarding

3. Answer Parsing Feature

After URL analysis, every free text answer in the remaining topics is parsed by Claude to extract data and identify missing information.

3.1 Answer Parsing Flow

Step 1: Merchant submits free text answer to any question

Step 2: Frontend sends answer to POST /api/onboarding/parse-answer

Step 3: Backend sends answer to Claude with topic-specific prompt

Step 4: Claude extracts all data points and identifies missing fields

Step 5: Frontend conditionally renders only follow-up questions for missing fields

3.2 Topic-Specific Parsing

Each topic requires a different Claude prompt:

- **Topic 1:** Extract storeURL, productCategory, foundingStory, geographicReach, fulfillmentModel
- **Topic 2:** Extract gender, ageRange, incomeLevel, interests, location, purchaseMotivation
- **Topic 3:** Extract storeAge, salesCount, marketingAttempted, resultsOfAttempts
- **Topic 4:** Extract weeklyHours, onCamera, socialMediaComfort, contentFormats
- **Topic 5:** Extract nineDayTarget, sideProjectOrMainIncome, successDefinition

4. Environment and Error Handling

4.1 Vercel Environment Variables

Store the Anthropic API key in Vercel: ANTHROPIC_API_KEY=sk-ant-xxxxx. Verify at startup. If missing, throw an error immediately.

4.2 Error Handling Strategy

If any Claude API call fails, gracefully degrade. For URL analysis, show all follow-ups. For answer parsing, assume all fields are missing. Never crash the onboarding.

4.3 Timeout Handling

URL fetches should timeout after 5 seconds. Claude API calls should timeout after 15 seconds. On timeout, return a user-friendly error and allow retry or skip.

4.4 Confidence Threshold

Claude returns a confidence score for parsing. If below 70 percent, show all follow-up questions as a fallback rather than risk extracting wrong data.

5. Testing Requirements

Before launch, test both URL analysis and answer parsing with real examples.

5.1 URL Analysis Test Cases

Test	URL Type	Expected
Valid Shopify store	Example Shopify site	Correctly identify products, positioning, price tier
WooCommerce store	Example WooCommerce site	Correctly identify platform and store details
New empty store	Mostly blank homepage	Detect early stage, minimal products
Invalid URL	Nonexistent domain	Gracefully handle with user-friendly error
Unreachable URL	Valid domain, server down	Timeout gracefully and allow retry

5.2 Answer Parsing Test Cases

Scenario	Answer	Expected Follow-ups
Complete detail	women 25-45 urban mid-upper fashion quality	None
Generic answer	women who like fashion	Ask: age, income, location
Partial answer	women 25-45 fashion	Ask: income, location, interests
Vague answer	my customers	Show all follow-ups

5.3 Real Store Testing

Before launch, test URL analysis against at least 10 real merchant stores across different niches: fashion, handmade, POD, dropship, digital. Verify Claude correctly identifies positioning, products, and price tier.

6. Timeline

This feature should take approximately 10-12 days to fully implement and test.

- Day 1-2: Set up URL fetching and HTML parsing library
- Day 3-4: Build store URL analysis endpoint and Claude prompt
- Day 5-6: Build answer parsing endpoint and topic-specific prompts
- Day 7-8: Frontend implementation for URL analysis and conditional rendering
- Day 9-10: Error handling, timeouts, graceful degradation
- Day 11-12: Testing with real stores and test cases, refinement

DELIVERABLE: Fully functional onboarding with URL analysis and intelligent answer parsing, comprehensive error handling, passing all test cases with 10 real stores before launch.